专题一：动态规划综合题题库（理论分析型）

题型说明

动态规划综合题的典型作答要求：

1. **分析最优子结构**：证明问题具有最优子结构性质
2. **定义状态**：给出dp数组的含义
3. **建立状态转移方程**：推导递推关系
4. **确定边界条件**：初始化和遍历顺序
5. **伪代码实现**：描述算法步骤
6. **复杂度分析**：时间复杂度和空间复杂度

题目1：矩阵链乘法

【问题描述】 给定n个矩阵的链 $\langle A_1, A_2, \ldots, A_n \rangle$，矩阵$A_i$的维度为$p_{i-1} \times p_i$。矩阵乘法满足结合律，但不同的加括号方式会导致不同的标量乘法次数。请设计动态规划算法，找到一种加括号方式，使得计算整个矩阵链乘积所需的标量乘法次数最少。

【输入示例】 矩阵维度：p = [30, 35, 15, 5, 10, 20, 25]（表示6个矩阵：30×35, 35×15, 15×5, 5×10, 10×20, 20×25）

【输出示例】 最少乘法次数：15125

【答题框架】

1. 最优子结构分析

引理：矩阵链乘问题具有最优子结构性质。

证明：设$A_i \ldots A_j$的最优加括号方式在A_k和A_{k+1}之间断开，即计算顺序为 $(A_i \ldots A_k) \times (A_{k+1} \ldots A_j)$。若存在另一种计算$A_i \ldots A_k$的方式比当前方式更优，则将其替换后能得到更优的$A_i \ldots A_j$计算方案，与假设矛盾。因此，$A_i \ldots A_k$的计算方式必然是最优的。同理$A_{k+1} \ldots A_j$也是最优的。

2. 状态定义

- 令 **m[i][j]** 表示计算矩阵链$A_i \times A_{i+1} \times \ldots \times A_j$所需的最少标量乘法次数（$1 \le i \le j \le n$）
- 令 **s[i][j]** 记录m[i][j]取最小值时的分割点k

3. 状态转移方程

边界条件：当 i = j 时，m[i][i] = 0（单个矩阵无需乘法）

递推关系：当 i < j 时

m[i][j] = min{ m[i][k] + m[k+1][j] + p[i-1] × p[k] × p[j] }，其中 i ≤ k < j

公式含义：将链在k处断开，先计算左半部分A[i..k]（代价m[i][k]），再计算右半部分A[k+1..j]（代价m[k+1][j]），最后将两个结果矩阵相乘（代价p[i-1]×p[k]×p[j]）。

4. 伪代码

text

```
算法 MatrixChainOrder(p, n)
// 输入：矩阵维度数组p[0..n]，n为矩阵个数
// 输出：最少乘法次数表m和分割点表s

1.  for i ← 1 to n do
2.      m[i][i] ← 0
3.  end for
4.
5.  for len ← 2 to n do                  // 链长度
6.      for i ← 1 to n - len + 1 do
7.          j ← i + len - 1
8.          m[i][j] ← ∞
9.          for k ← i to j - 1 do
10.             q ← m[i][k] + m[k+1][j] + p[i-1] * p[k] * p[j]
11.             if q < m[i][j] then
12.                 m[i][j] ← q
13.                 s[i][j] ← k
14.             end if
15.         end for
16.     end for
17. end for
18.
19. return m, s
```

5. 时间复杂度分析

- 三层循环：外层len循环 O(n)，中层i循环 O(n)，内层k循环 O(n)
- 总时间复杂度：**$O(n^3)$**
- 空间复杂度：**$O(n^2)$**（存储m和s两个表）

题目2：最长公共子序列（LCS）

【问题描述】 给定两个序列$X = \langle x_1, x_2, ..., x_m \rangle$和$Y = \langle y_1, y_2, ..., y_n \rangle$，找出X和Y的最长公共子序列的长度。子序列不要求连续，但顺序必须一致。

【输入示例】 X = "ABCBDAB", Y = "BDCABB"

【输出示例】 最长公共子序列长度为4（如"BCAB"或"BDAB"）

【答题框架】

1. 最优子结构分析

设$X_i = \langle x_1, x_2, \ldots, x_i \rangle$，$Y_j = \langle y_1, y_2, \ldots, y_j \rangle$，LCS的长度为c[i][j]。则有以下性质：

- **性质1**：若 $x_i = y_j$，则X_i和Y_j的LCS等于X_{i-1}和Y_{j-1}的LCS加上x_i
- **性质2**：若 $x_i \neq y_j$，则X_i和Y_j的LCS等于 max(LCS(X_{i-1}, Y_j), LCS(X_i, Y_{j-1}))

证明：

- 若$x_i = y_j$，假设存在更长的公共子序列不包含x_i，则可以在该子序列末尾加上x_i得到更长序列，矛盾。
- 若$x_i \neq y_j$，则最长公共子序列要么来自X_{i-1}与Y_j（不含x_i），要么来自X_i与Y_{j-1}（不含y_j），取两者最大值。

2. 状态定义

c[i][j] = X[1..i]与Y[1..j]的最长公共子序列长度（$0 \le i \le m$，$0 \le j \le n$）

3. 状态转移方程

边界条件：
c[0][j] = 0（空序列与任何序列的LCS长度为0）
c[i][0] = 0（任何序列与空序列的LCS长度为0）

递推关系：

情况1：当 i > 0, j > 0 且 x[i] = y[j] 时
c[i][j] = c[i-1][j-1] + 1

情况2：当 i > 0, j > 0 且 x[i] ≠ y[j] 时
c[i][j] = max(c[i-1][j], c[i][j-1])

4. 伪代码

text

```
算法 LCS_Length(X, Y, m, n)
// 输入：序列X[1..m]，Y[1..n]
// 输出：LCS长度表c和方向表b

1.  for i ← 0 to m do
2.      c[i][0] ← 0
3.  end for
4.  for j ← 0 to n do
5.      c[0][j] ← 0
6.  end for
```

```
7.
8.  for i ← 1 to m do
9.      for j ← 1 to n do
10.         if X[i] = Y[j] then
11.             c[i][j] ← c[i-1][j-1] + 1
12.             b[i][j] ← "↖"
13.         else if c[i-1][j] ≥ c[i][j-1] then
14.             c[i][j] ← c[i-1][j]
15.             b[i][j] ← "↑"
16.         else
17.             c[i][j] ← c[i][j-1]
18.             b[i][j] ← "←"
19.         end if
20.     end for
21. end for
22.
23. return c, b
```

5. 时间复杂度分析

- 两层循环：外层i循环 O(m)，内层j循环 O(n)
- 总时间复杂度：**O(m × n)**
- 空间复杂度：**O(m × n)**（可优化至O(min(m, n))）

题目3：0-1背包问题

【问题描述】 有n个物品和一个容量为C的背包。每个物品i有重量w[i]和价值v[i]。每个物品只能选择0个或1个。问如何选择物品，使得总重量不超过C的情况下总价值最大。

【输入示例】
n = 4, C = 8
物品：重量=[2, 3, 4, 5]，价值=[3, 4, 5, 6]

【输出示例】 最大价值：9（选择物品1和2，总重3+4=7≤8，价值4+5=9）

【答题框架】

1. 最优子结构分析

设最优解为S，物品n要么在S中，要么不在。

- 若 n ∈ S，则 S - {n} 是物品{1..n-1}在容量 C - w[n] 下的最优解
- 若 n ∉ S，则 S 是物品{1..n-1}在容量 C 下的最优解

证明：假设n ∈ S，若存在另一个解T是物品{1..n-1}在容量C-w[n]下的更优解（价值更大），则 T ∪ {n} 是原问题的一个更优解，与S是最优解矛盾。因此S-{n}必然是最优子问题的解。

2. 状态定义

令 **dp[i][j]** 表示考虑前i个物品（1..i），背包容量为j时的最大总价值。

3. 状态转移方程

边界条件:
dp[0][j] = 0（没有物品可选，价值为0）
dp[i][0] = 0（容量为0，无法放任何物品）

递推关系:

情况1：当 j < w[i] 时（容量不足以放入物品i）
dp[i][j] = dp[i-1][j]

情况2：当 j ≥ w[i] 时
dp[i][j] = max(dp[i-1][j], dp[i-1][j-w[i]] + v[i])

公式解释:

- 不选物品i：价值为 dp[i-1][j]
- 选物品i：价值为 dp[i-1][j-w[i]] + v[i]（前提是容量足够）

4. 伪代码（空间优化版）

text

```
算法 Knapsack_01(w, v, C, n)
// 输入：重量数组w[1..n]，价值数组v[1..n]，容量C
// 输出：最大总价值

1.  dp[0..C] ← 0
2.
3.  for i ← 1 to n do
4.      for j ← C downto w[i] do    // 逆序遍历，保证每个物品只选一次
5.          dp[j] ← max(dp[j], dp[j - w[i]] + v[i])
6.      end for
7.  end for
8.
9.  return dp[C]
```

说明：逆序更新是为了保证dp[j - w[i]]是上一轮（未考虑物品i）的结果，避免同一物品被多次选择。

5. 时间复杂度分析

- 两层循环：外层i循环 O(n)，内层j循环 O(C)
- 总时间复杂度：**O(n × C)**
- 空间复杂度：优化后为 **O(C)**，优化前为 **O(n × C)**

注：这是伪多项式时间复杂度，当C很大时算法效率较低。

题目4：石子合并问题

【问题描述】 在一条直线上有n堆石子，每堆有a[i]个石子。每次只能合并相邻的两堆，合并代价为两堆石子数之和。经过n-1次合并后，所有石子合并成一堆。求最小总合并代价。

【输入示例】 n=4，石子数=[4, 2, 3, 5]

【输出示例】 最小总代价：28

1. 最优子结构分析

设将第i堆到第j堆石子合并成一堆的最小代价为dp[i][j]。最后一次合并必然是将[i..k]堆和[k+1..j]堆合并，代价为sum(i,j)（即第i到j堆的石子总数）。因此，最优解包含子问题[i..k]和[k+1..j]的最优解。

2. 状态定义

- **dp[i][j]**：合并第i堆到第j堆石子所需的最小总代价（1 ≤ i ≤ j ≤ n）
- **sum[i][j]**：第i堆到第j堆的石子总数，可用前缀和快速计算

前缀和计算公式：
prefix[0] = 0
prefix[t] = prefix[t-1] + a[t] (t = 1..n)
sum(i,j) = prefix[j] - prefix[i-1]

3. 状态转移方程

边界条件：
当 i = j 时，dp[i][i] = 0（只有一堆，无需合并）

递推关系：当 i < j 时
dp[i][j] = min{ dp[i][k] + dp[k+1][j] + sum(i,j) }，其中 i ≤ k < j

4. 伪代码

text

```
算法 StoneMerge(a, n)
// 输入：石子数组a[1..n]，n堆石子
// 输出：最小合并总代价

1.  prefix[0] ← 0
2.  for i ← 1 to n do
3.      prefix[i] ← prefix[i-1] + a[i]
```

```
4.  end for
5.
6.  for i ← 1 to n do
7.      dp[i][i] ← 0
8.  end for
9.
10. for len ← 2 to n do                    // 区间长度
11.     for i ← 1 to n - len + 1 do
12.         j ← i + len - 1
13.         dp[i][j] ← ∞
14.         for k ← i to j - 1 do
15.             cost ← dp[i][k] + dp[k+1][j] + (prefix[j] - prefix[i-1])
16.             if cost < dp[i][j] then
17.                 dp[i][j] ← cost
18.             end if
19.         end for
20.     end for
21. end for
22.
23. return dp[1][n]
```

5. 时间复杂度分析

- 三层循环：外层len O(n)，中层i O(n)，内层k O(n)
- 总时间复杂度：**$O(n^3)$**
- 空间复杂度：**$O(n^2)$**

注：可使用四边形不等式优化到$O(n^2)$

题目5：编辑距离（Levenshtein Distance）

【问题描述】 给定两个字符串A和B，允许三种操作：插入一个字符、删除一个字符、替换一个字符。求将A转换为B所需的最少操作次数。

【输入示例】 A = "horse", B = "ros"

【输出示例】 最少操作次数：3

- horse → rorse（替换h为r）
- rorse → rose（删除r）
- rose → ros（删除e）

【答题框架】

1. 最优子结构分析

设dp[i][j]表示将A[1..i]转换为B[1..j]的最小操作次数。考虑A[i]和B[j]：

- 若A[i] = B[j]，则无需额外操作，问题转化为A[1..i-1]转换为B[1..j-1]
- 若A[i] ≠ B[j]，则可以通过三种操作之一：
  - 替换：将A[i]替换为B[j]，操作数+1，子问题为A[1..i-1]转换为B[1..j-1]
  - 删除：删除A[i]，操作数+1，子问题为A[1..i-1]转换为B[1..j]
  - 插入：在A[i]后插入B[j]，操作数+1，子问题为A[1..i]转换为B[1..j-1]

2. 状态定义

dp[i][j]：A[1..i]转换为B[1..j]的最小编辑距离（0 ≤ i ≤ m，0 ≤ j ≤ n）

3. 状态转移方程

边界条件：

- dp[0][j] = j（将空串转换为B[1..j]需要j次插入）
- dp[i][0] = i（将A[1..i]转换为空串需要i次删除）

递推关系：当 i > 0 且 j > 0 时

情况1：若 A[i] = B[j]
dp[i][j] = dp[i-1][j-1]

情况2：若 A[i] ≠ B[j]
dp[i][j] = 1 + min(dp[i-1][j-1], dp[i-1][j], dp[i][j-1])
其中：

- dp[i-1][j-1] 对应替换操作
- dp[i-1][j] 对应删除操作
- dp[i][j-1] 对应插入操作

4. 伪代码

text

```
算法 EditDistance(A, B, m, n)
// 输入：字符串A[1..m]，B[1..n]
// 输出：最小编辑距离

1.  for i ← 0 to m do
2.      dp[i][0] ← i
3.  end for
4.  for j ← 0 to n do
5.      dp[0][j] ← j
6.  end for
7.
8.  for i ← 1 to m do
9.      for j ← 1 to n do
10.         if A[i] = B[j] then
```

```
            dp[i][j] ← dp[i-1][j-1]
        else
            dp[i][j] ← 1 + min(dp[i-1][j-1], dp[i-1][j], dp[i][j-1])
        end if
    end for
end for

return dp[m][n]
```

5. 时间复杂度分析

- 两层循环：外层i O(m)，内层j O(n)
- 总时间复杂度：**O(m × n)**
- 空间复杂度：**O(m × n)**（可优化至O(min(m, n))）

动态规划复杂度总结表

问题	状态定义	状态数	转移代价	总时间复杂度	空间复杂度
矩阵链乘	m[i][j]	$O(n^2)$	O(n)	$O(n^3)$	$O(n^2)$
LCS	c[i][j]	O(mn)	O(1)	O(mn)	O(mn)
0-1背包	dp[i][j]	O(nC)	O(1)	O(nC)	O(C)
石子合并	dp[i][j]	$O(n^2)$	O(n)	$O(n^3)$	$O(n^2)$
编辑距离	dp[i][j]	O(mn)	O(1)	O(mn)	O(mn)

题目0：斐波那契数列（动态规划入门案例）

【问题描述】 斐波那契数列定义为：

- F(0) = 0
- F(1) = 1
- F(n) = F(n-1) + F(n-2)，当 n ≥ 2

给定非负整数n，求F(n)的值。

【输入示例】 n = 10

【输出示例】 F(10) = 55

【答题框架】

1. 最优子结构分析

引理：斐波那契数列具有最优子结构性质。

证明：F(n) 的解依赖于 F(n-1) 和 F(n-2) 的解。要计算 F(n)，必须先知道 F(n-1) 和 F(n-2) 的值。若存在更优的方式计算 F(n-1) 或 F(n-2)，则直接替换后能得到更优的 F(n)，但斐波那契数列的值是唯一确定的，因此子问题的解必然是最优的（也是唯一的）。✅

重叠子问题：递归计算时，F(n-2) 会被多次重复计算。例如计算 F(5) 时：

- F(5) 调用 F(4) 和 F(3)
- F(4) 调用 F(3) 和 F(2)
- F(3) 被计算了2次

这正是动态规划要优化的地方——用空间换时间，避免重复计算。

2. 状态定义

令 **dp[i]** 表示斐波那契数列第 i 项的值，即 F(i)。

3. 状态转移方程

边界条件：

- dp[0] = 0
- dp[1] = 1

递推关系：当 i ≥ 2 时
dp[i] = dp[i-1] + dp[i-2]

4. 三种实现方式的对比

方式一：朴素递归（不推荐，指数级复杂度）

text

```
算法 Fib_Recursion(n)
// 输入：非负整数n
// 输出：第n项斐波那契数

1. if n = 0 then return 0
2. if n = 1 then return 1
3. return Fib_Recursion(n-1) + Fib_Recursion(n-2)
```

时间复杂度：$O(2^n)$（递归树节点数约为2^n）
空间复杂度：O(n)（递归栈深度）

方式二：动态规划（自底向上，推荐）

text

```
算法 Fib_DP(n)
// 输入：非负整数n
// 输出：第n项斐波那契数

1. if n = 0 then return 0
2. if n = 1 then return 1
3.
4. dp[0..n] ← 新数组
5. dp[0] ← 0
6. dp[1] ← 1
7.
8. for i ← 2 to n do
9.      dp[i] ← dp[i-1] + dp[i-2]
10. end for
11.
12. return dp[n]
```

时间复杂度：O(n)（单层循环）
空间复杂度：O(n)（存储dp数组）

方式三：空间优化版（只保留前两个值）

text

```
算法 Fib_Optimized(n)
// 输入：非负整数n
// 输出：第n项斐波那契数

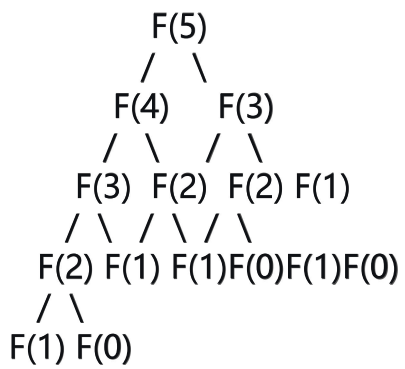
1.  if n = 0 then return 0
2.  if n = 1 then return 1
3.
4.  prev2 ← 0    // F(0)
5.  prev1 ← 1    // F(1)
6.
7.  for i ← 2 to n do
8.      curr ← prev1 + prev2
9.      prev2 ← prev1
10.     prev1 ← curr
11. end for
12.
13. return prev1
```

时间复杂度：O(n)（单层循环）
空间复杂度：O(1)（只使用三个变量）

5. 递归树分析（说明重叠子问题）

以 n = 5 为例，递归调用树如下：

```
text
```



观察：

- F(3) 被计算了2次
- F(2) 被计算了3次
- F(1) 被计算了5次
- F(0) 被计算了3次

随着 n 增大，重复计算的次数呈指数级增长。动态规划通过存储已计算结果，将指数级复杂度降为线性。

6. 时间复杂度对比

实现方式	时间复杂度	空间复杂度	适用场景
朴素递归	$O(2^n)$	O(n)	$n \le 30$
动态规划（数组）	O(n)	O(n)	需要保留所有中间结果
动态规划（空间优化）	O(n)	O(1)	只需要第n项的值
矩阵快速幂	O(log n)	O(1)	超大n（如$n=10^9$）

7. 拓展思考

问题1：为什么朴素递归的效率那么低？

答案：因为存在大量重叠子问题。每次递归调用都会重新计算子问题的值，而不是复用之前计算的结果。

问题2：动态规划相比朴素递归的改进是什么？

答案：用空间换时间。通过存储已经计算过的子问题结果，避免重复计算，将指数级时间复杂度降为线性。

问题3：如果n非常大（如 $n = 10^9$），O(n) 的算法仍然太慢，该怎么办？

答案：可以使用矩阵快速幂方法，将时间复杂度降为 O(log n)。核心公式：

text

```
[ F(n)   F(n-1) ]   =   [ 1   1 ] ^ (n-1)
[ F(n-1) F(n-2) ]       [ 1   0 ]
```